

# Amortized Deduction for Binary Linear Systems: Learning Branching Guidance from Exact Conditional Marginals

Binary linear system을 위한 amortized deduction:  
정확한 conditional marginal로부터 branching guidance를 학습하기  
neuroMCTS project · 2026년 9월 · 한국어판

## 초록

Binary linear system(BLS)은  $A \in \{0,1\}^{m \times n}$ 에 대해  $Ax=b$ 를 만족하는  $x \in \{0,1\}^n$ 를 찾는 문제이며, 레이저 기반 비파괴검사에서 정수 투영값으로부터 이진 점유 영상을 복원하는 과제로 나타난다. market-split 계열 인스턴스에서는 linear relaxation이 의도적으로 무정보하도록 설계되므로, backtracking search는 값싼 heuristic으로는 잘 내릴 수 없는 branching 결정에 대부분의 노력을 쓴다. 본 논문은 이 설정을 위한 학습 기반 branching heuristic과, 그것을 지도학습하기 위한 정확한 절차를 제시한다.

방법은 두 관찰에 기반한다. 첫째, BLS를 partial assignment로 conditioning하는 것은 그 배정을 대입해 더 작은 BLS를 얻는 것과 동일하다. 따라서 하나의 network가 구조 변경 없이 root와 모든 interior node에서 예측하며, 학습 상태는 크기가 줄어든 평범한 instance가 된다. 둘째, 전수열거가 가능한 크기의 instance에서는 정확한 conditional marginal  $p_j = \Pr[x_j=1 \mid A, b]$ 를 계산할 수 있고, 이것이 node의 자유 변수를 두 부류로 분할한다: 살아남은 모든 해가 일치해 반대로 분기하면 subtree가 비어 backtrack이 확정되는 변수(DET)와, 해들이 갈려 어느 쪽으로 분기해도 해에 도달하는 변수(OPEN)다. 비용을 발생시키는 것은 전자뿐이다. 이 분할은 정확한 학습 target과 동시에 어떤 예측이 중요한지에 대한 기준을 제공한다: root에서는 변수의 6.3%만이 DET이지만 depth 8에서는 그 비중이 89.0%가 되며, 정확도가 탐색 비용으로 환산되기 시작하는 지점이 바로 여기다.

우리는 21.8K 파라미터의 bipartite graph network를 이 target에 학습시키고, 오직 branching에만 사용한다 — 증명이 아니라 확률을 내놓으므로 변수 확정에는 쓰지 않는다. depth 5~8의 DET 변수에서 instance 전체 판정 1회당 1.6ms에 88~91% 정확도를 얻는 반면, 변수당 linear program을 한 번씩 지불하는 LP-probing은 31~37ms에 61~80% 탐지율을 보인다. 즉 학습된 heuristic은 같거나 더 나은 적중률에서 19~23배 저렴하며, 이는 classical reasoning이 닿을 수 없는 정보의 발견이 아니라 amortized optimization의 사례다. wall-clock 예산을 맞춘 비교에서 21×60 기준 LP 기반 branching 대비 median 2.71배 빠르고(sign test  $p=0.019$ ), node를 6.2배 적게 전개하며, 해결율을 90.0%에서 100%로 올린다. lattice reduction이 먼저 도는 전체 pipeline에서는 그 단계가 이 크기에서 16/30을 닫고, guidance가 end-to-end 해결율을 93.3%에서 100%로 끌어올린다. solution 개수를 크기 간에 통제하면 transfer 손실이 측정되지 않는다: 10×25에서만 학습한 network가 평가 크기에서 학습한 network와 통계적으로 구별되지 않는다. 코드, 생성기, 정확한 marginal label, 인스턴스 단위 결과를 공개한다.

## 1. 서론

### 1.1 문제

$A \in \{0,1\}^{m \times n}$ ,  $b \in \mathbb{Z}^m$ 이 주어질 때 binary linear system(BLS)은 다음 feasibility 문제다:

$$Ax = b \text{ 를 만족하는 } x \in \{0,1\}^n \text{ 를 찾아라. (1)}$$

그 해집합을  $S = \{x \in \{0,1\}^n : Ax=b\}$ 라 쓴다. 각 row는 cardinality constraint로, 지정된 셀 부분집합 중 몇 개가 점유되어 있는지를 나타낸다.

응용 배경은 레이저 기반 비파괴검사다. 시편을  $m$ 개 방향으로 스캔하면 각 스캔은 경로상 흡수 셀의 정수 개수를 반환하고, 과제는 이진 점유 영상을 복원하는 것이다. 이 설정의 두 성질이 문제 정식화를 규정한다. 해는 유일할 필요가 없다 — 동일한 측정치와 부합하는 영상이 여럿일 수 있고,  $S$ 의 아무 원소나 복원하면 물리적 문제는 해결된다. 그리고 측정 노이즈가  $b$ 를 교란해  $S=\emptyset$ 이 될 수 있으므로, 신뢰할 수 있는 infeasibility 판정도 해를 찾는 것만큼 가치가 있다.

$S \neq \emptyset$  판정은 일반적으로 NP-complete이다. 본 연구의 인스턴스는 market-split 문제를 본뜬 고난도 영역에서 의도적으로 추출되며, 그 영역에서는 linear relaxation  $S_{LP} = \{x \in [0,1]^n : Ax=b\}$ 가 대부분 fractional한 vertex를 갖는 큰 polytope이다. 따라서 relaxation 기반 bounding이 약하고, complete solver는 relaxation의 도움을 거의 받지 못한 채 어디서 분기할지 결정해야 한다 — 이것이 본 논문이 학습하는 결정이다.

### 1.2 학습 컴포넌트가 붙는 자리, 그리고 무엇으로 학습할 것인가

식 (1)을 위한 complete solver는 deduction(constraint propagation, relaxation 기반 가지치기)과 branching 결정(어느 자유 변수로 분기하고 어느 값을 먼저 시도할 것인가)을 번갈아 수행한다. deduction은 건전해야 하고 앞으로도 그래야 하지만, branching은 heuristic한 선택이라 오류의 대가가 탐색 시간일 뿐 정확성이 아니다. 이 비대칭이 branching을 학습 컴포넌트가 붙기에 자연스러운 자리로 만들며, mixed-integer programming에서 학습이 가장 성공한 지점이기도 하다.

어려운 것은 지도(supervision)다. 표준적인 답이 두 가지 있는데 여기서는 둘 다 만족스럽지 않다. 심어진 해  $x^*$ 로 학습하는 것은  $S$ 의 한 원소를 ground truth로 취급하는데,  $|S|>1$ 이면 이것은 잘못 정의된 문제다:  $(A,b)$ 의 어떤 것도 심어진 해를 다른 해와 구분하지 못하므로 label은 posterior가 아니라 posterior로부터의 표본이다. strong-branching imitation처럼 비싼 expert를 모방하는 것은 그 결정이 유익하면서도 수집 가능한 expert를 요구하는데, 여기서 자연스러운 후보인 LP-probing은 node당 변수당 linear program 한 번을 지불한다.

우리는 세 번째 길을 택한다. 이 인스턴스들이 planted solution을 갖고 적당한 크기에서 feasibility를 열거할 수 있기 때문에 가능한 길이다: 작은 인스턴스에서 전수열거로 정확한 conditional posterior를 계산하고 그것에 학습한다. §3은 이것이 실행 가능하며, 정확한 의미에서 올바른 target임을 보인다 — 그리고 그 posterior가 어떤 예측이 탐색에 영향을 줄 수 있는지까지 함께 식별함을 보인다.

### 1.3 기여

(1) in-tree 지도학습을 간단히 만드는 reduction 항등식. 식 (1)을 partial assignment로 conditioning하면 같은 구조의 더 작은 BLS가 된다(명제 1). 따라서 하나의 architecture, 하나의 label 생성기, 하나의 학습 루프가 root와 모든 interior node를 담당하며, depth- $d$  학습 상태는  $n-d$ 개 변수를 갖는 평범한 instance로 저장된다.

(2) 비용을 발생시킬 수 있는가에 따른 branching 결정의 분할. 정확한 marginal이 node의 자유 변수를 DET(모든 살아남은 해가 일치; 반대로 분기하면 subtree가 빔)와 OPEN(해들이 갈림; 어느 쪽이든 해에 도달)으로 나눈다. DET 오류만이 backtrack을 유발한다. 그 구성을 측정하면 root에서 6.3%, depth 8에서 89.0%다 (§4.3, §4.4). 이는 쓸모 있는 예측이 root가 아니라 tree 내부에 있음을 특정하며, root 수준 정확도가 왜 나쁜 목적함수인지 설명한다.

(3) 비싼 건전 절차를 amortize하는 학습 heuristic. depth 5~8의 DET 변수에서 network는 node당 1.6ms에 88~91% 정확도를, LP-probing은 31~37ms에 61~80% 탐지율을 보인다 (§4.5). network는 guidance에만 쓰이며, 건전한 확정은 propagation과 probing의 몫으로 남는다. 이득은 도달 범위가 아니라 비용에 있다.

(4) wall-clock 예산을 맞춘 종단 평가.  $21 \times 60$ 에서 학습된 guidance는 LP 기반 branching 대비 median 2.71배 빠르고(sign test  $p=0.019$ ), node를 6.2배 적게 전개하며, 600초 내 30/30 대 27/30을 해결한다( § 4.6). 실제 cascade에서는 end-to-end 해결율이 93.3%에서 100%로 오른다( § 4.8).

(5) solution multiplicity를 통제한 크기 transfer.  $|S|$ 를 크기 간에 비슷하게 유지하면,  $10 \times 25$ 에서만 학습한 network가 평가 크기  $21 \times 60$ 에서 학습한 network와 대등하다(30개 중 15개에서 더 빠르고 총 시간 차 0.4%). 학습된 양이 크기에 특수하지 않음을 보인다( § 4.7).

## 2. 선행연구

### 2.1 Binary linear system과 market-split의 난이도

식 (1)은 0/1 integer feasibility 문제이며 일반 A에 대해 NP-complete이다. 그러나 실무적 난이도는 instance 분포에 달려 있다. Cornuéjols와 Dawande(1999)는 market-split 계열을 도입했다:  $m$ 명의 agent가  $n$ 개 품목을 각자의 예산에 정확히 맞게 나눠야 하며,  $n \approx 10(m-1)$ 일 때 놀랍도록 작은 크기에서 branch-and-bound를 무력화한다. 메커니즘은 integrality gap이다: relaxation polytope이 크고 그 vertex가 fractional이라 bounding이 거의 가지치기를 못 하고, 탐색은 사실상 눈먼 채로 분기해야 한다. Aardal 등(2000)은 lattice basis reduction이 같은 계열을 branch-and-bound보다 훨씬 효과적으로 공략함을 보였고, 우리는 § 3.2에서 이를 활용한다.

우리 생성기(§ 4.1)는 relaxation vertex들의 퍼짐(spread)을 최대화해 이 메커니즘을 명시적으로 만든다. 이는  $S_{LP}$ 를 키우고 relaxation signal을 설계 단계에서 악화시킨다.

### 2.2 Complete solver

Mixed-integer programming, linear relaxation 위의 branch-and-bound를 presolve와 cutting plane으로 강화하는 것이 표준이다. 그 지렛대는 relaxation이 infeasible하거나 target에서 멀리 떨어진 node를 가지치는 데서 나오는데, market-split 구성이 제거하는 것이 정확히 그것이다.

Clause learning을 갖춘 constraint programming. CP-SAT 계열 solver는 constraint propagation에 conflict-driven clause learning(CDCL)과 lazy clause generation을 결합한다. 이 계열에서의 강점은 본 논문의 논지에 핵심적인 성질에서 나온다: 탐색은 내려가면서 정보를 만들어낸다.  $\Pr[x_j=1 \mid A, b]$ 가 root에서 무정보해도  $\Pr[x_j=1 \mid A, b, x_1=1, x_5=0]$ 은 0이나 1일 수 있다. root에서 보이지 않던 구조가 commitment 이후 연역 가능해지며, 이것이 § 3.4가 root가 아니라 tree 내부를 다루는 이유다.

Lattice reduction. 대안적 접근은 식 (1)을 lattice에 embedding하고 짧은 vector를 찾는다. LLL은 다항시간에 basis를 축소하고, BKZ는 block size로 시간과 품질을 교환하며 이를 일반화한다. Aardal–Hurkens–Lenstra(AHL) embedding은 0/1 해가 짧은 vector로 직접 나타나도록 구성되어, reduction이 heuristic이 아니라 certificate를 생산하는 절차가 된다. 우리는 이를 전체 pipeline의 첫 단계로 사용한다.

### 2.3 조합최적화를 위한 학습

Bengio 등(2021)은 이 분야를 개관하며 우리가 채택하는 표현을 쓴다: solver는 "계산하기에 너무 비싸거나 수학적으로 잘 정의되지 않은 결정"에 대해 수작업 heuristic에 의존하며, 학습이 그 결정을 개선할 후보라는 것이다. too expensive to compute라는 표현은 그 결정이 원리적으로 계산 가능함을 전제하고 비용만을 묻는다 — 이것이 § 4.5에서 우리가 측정하는 상황이다.

Branching imitation. 이 분야의 가장 분명한 성공은 비싼 expert의 모방이다. strong branching은 두 자식의 relaxation을 잠정적으로 풀어 후보에 점수를 매기며, 작은 tree를 만들지만 매 node에서 돌리기에는 너무 느리다. Gasse 등(2019)은 bipartite graph network에 이를 모방시켜 훨씬 적은 비용으로 이득 대부분을 회수했다. 이득은 새 정보가 아니라 — expert는 이미 답을 알고 있었다 — 그 지식에 값싸게 접근할 수 있게 된 것이다. 우리 방법은 같은 구조적 위치를 차지하되 두 가지가 다르다: target이 expert의 점수가 아니라 정확한 posterior이며, 분석이 expert의 결정 중 어느 것이 중요한지를 식별한다.

직접적 satisfiability 예측. network가 satisfiability를 종단으로 판정하게 하려는 시도는 기록이 더 약하다. NeuroSAT은 literal-clause graph 위의 message passing으로 분류하며 작은 random 인스턴스에서 성공하지만, G4SATBench는 크기 transfer에서 상당한 정확도 붕괴를 보고한다. Chen 등(2023)은 표준 message-passing network가 어떤 feasible/infeasible MILP 쌍을 원리적으로 구분할 수 없음을 증명했다. 따라서 우리는 network에게 feasibility 판정을 요구하지 않는다. network는 건전성을 책임지는 complete search 내부에서 branching guidance만 제공한다.

### 2.4 Amortized optimization

우리 결과를 조직하는 구분은 조합최적화에서 쓰이기 전부터 있었다. Gershman과 Goodman(2014)은 amortized inference를 도입했다: 관련된 query가 많을 때 매번 처음부터 푸는 것은 낭비이며, 질문은 query 간에 계산을 어떻게 재사용하는가가 된다. Amos(2023)는 같은 아이디어를 amortized optimization으로 전개한다 — 유사한 problem instance 간의 공유 구조를 활용해 학습으로 해를 예측하는 것 — 그리고 variational inference, meta-learning, control에서의 등장을 개관한다.

Millidge(2022)는 그 trade-off를 여기서 가장 관련 있는 형태로 진술한다: direct optimization은 당면 instance에 계산을 쓰며 호출할 때마다 그 비용을 지불하고, amortized optimization은 문제를 supervised learning으로 바꿔 inference를 값싸게 만들지만 풀린 인스턴스 corpus가 선결 조건이고 approximator의 generalization에 갇힌다. 생산적인 배치는 hybrid다 — amortized 예측이 direct optimizer를 안내한다.

우리 시스템이 그 hybrid를 정확히 구현한다. direct 방법은 LP-probing이며 건전하되 비싸다. amortized 방법은 network이며 값싸되 건전하지 않다. hybrid 규칙은 network가 branching을 안내하되 probing과 propagation이 변수 확정에 대한 배타적 권한을 유지하는 것이다 (§ 3.7).

### 3. 제안 방법

#### 3.1 설정과 표기

변수 집합  $F$ 를 값  $v$ 로 고정하는 partial assignment에 대해, 그와 부합하는 해집합을  $S(F,v) = \{x \in S : x_F = v\}$ 라 쓰고, 자유 변수 집합을  $K$ 라 한다. 자유 변수  $j \in K$ 의 conditional marginal은 다음과 같다:

$$p_j(F,v) = |\{x \in S(F,v) : x_j = 1\}| / |S(F,v)| \quad (2)$$

#### 3.2 시스템 개요

실제 solver는 lattice 단계와 guided complete search의 cascade다:

propagation  $\rightarrow$  LP infeasibility rule  $\rightarrow$  kernel pump  $\rightarrow$  AHL/BKZ  $\rightarrow$  guided backtracking search

앞의 세 단계는 값싸고 건전하다. AHL/BKZ는 크기 의존적 block 파라미터로 certificate를 생산하는 lattice reduction을 시도하며, 성공하면 검증된 해를 바로 반환한다. 학습 컴포넌트는 마지막 단계에만, 그것도 branching 결정에만 나타난다. 이 배치는 평가에서 중요하다: lattice 단계가 branching heuristic과 무관하므로 비교 대상인 모든 heuristic이 정확히 같은 잔여 instance 집합을 받으며, 따라서 그 잔여 위에서 조건부로 보는 것은 유리한 사례를 고르는 것이 아니라 알고리즘을 기술하는 것이다.

#### 3.3 Conditioning은 reduction이다

명제 1.  $A_F$ 와  $A_K$ 를 각각  $F, K$ 로 인덱싱된 열 부분행렬이라 하자. 그러면

$$S(F,v) \cong \{y \in \{0,1\}^{|K|} : A_K y = b - A_F v\} \quad (3)$$

이며 대응은  $x \mapsto x_K$ 이다. 따라서 원본 instance의 conditional marginal (2)는 축소된 instance의 통상적 marginal이다.

증명.  $Ax = A_F x_F + A_K x_K$ 이다.  $x_F = v$ 로 고정하면  $Ax = b \iff A_K x_K = b - A_F v$ 이고,  $x_F$ 가 결정되어 있으므로  $x \mapsto x_K$ 는 두 집합 사이의 전단사다. marginal에 대한 진술은 개수를 세면 따라 나온다.  $\square$

이 항등식은 초등적이지만, in-tree 지도학습을 실행 가능하게 만드는 것이 바로 이것이다. depth-d search node는 자기만의 encoding을 요구하는 특수한 대상이 아니라, 같은 문제의  $n-d$ 개 변수 instance일 뿐이다. 따라서 하나의 network, 하나의 feature 추출기, 하나의 label 생성기가 tree 전체를 담당하며, depth d의 학습 데이터는 solver를 계측하는 것이 아니라 대입으로 생산된다.

두 구현 세부가 정합성을 유지한다. 대입 후 전부 0이 된 row는 제약을 부과하지 않으므로 제거한다 — 남겨두면 linear programming backend가 모델을 거부한다. 그리고 자유 bound 조건  $0 \leq b - A_F v \leq A_K \mathbf{1}$ 을 relaxation을 풀기 전에 검사한다. 무시할 만한 비용으로 dead node의 상당 부분을 걸러내기 때문이다.

### 3.4 어떤 예측이 비용을 발생시킬 수 있는가

살아남은 해집합이 비어 있지 않은 node에서, 자유 변수를 conditional marginal로 분할한다:

$$DET = \{j : p_j \in \{0,1\}\}, \quad OPEN = \{j : 0 < p_j < 1\}$$

이 구분은 통계적이 아니라 운용적이다.  $j \in OPEN$ 이면  $x_j$ 로 분기한 두 자식 모두 해를 포함하므로, 어느 쪽을 골라도 해가 도달 가능한 채로 남는다 — 즉 OPEN 변수에서의 어떤 branching 결정도 비용을 발생시킬 수 없다.  $j \in DET$ 이면 한쪽 자식이 비어 있다. 강제된 값을 거슬러 분기하면 subtree가 해 없이 소진되는 것이 확정되며, 이는 온전한 backtrack 비용이다.

이것이 conditional marginal의 운용적 독법을 준다. 그것은 지정된 어떤 해에 대한 "맞힐 확률"이 아니라 그 배정으로 살아남은 해집합의 비율이며, 중요한 오류는 그 비율이 0이거나 1인 변수에서의 오류뿐이다.

두 가지 귀결이 따르고, 실험이 둘 다 확인한다. 첫째, 모든 자유 변수에 대한 총계 정확도는 나쁜 목적함수다. backtrack을 유발할 수 있는 결정과 그럴 수 없는 결정을 섞기 때문이다. 관련 있는 양은 DET에서의 정확도다. 둘째, 탐색이 내려갈수록  $|S(F,v)|$ 가 줄어들므로 DET 비중이 depth에 따라 커지며, 쓸모 있는 예측은 root가 아니라 tree 내부에 집중된다.

### 3.5 학습 target으로서의 정확한 conditional marginal

전수열거가 가능한 크기의 instance에서는 constraint solver의 all-solutions 모드로 S를 정확히 얻고, 식 (2)를 개수로 계산한다. 학습은 그 결과 얻어지는 Bernoulli target에 대한 cross-entropy를 최소화한다:

$$L = -(1/|K|) \sum_j [p_j \log p_j^\wedge + (1-p_j) \log(1-p_j^\wedge)] \quad (4)$$

이는 정확히  $p_j^\wedge = p_j$ 에서 최소화된다.  $p_j$  자리에 심어진 해를 넣는 관례적 선택은  $|S| > 1$ 일 때마다 잡음 섞인 대리 target이 된다. 심어진 해는 posterior에서 뽑은 한 표본이며, 우리 인스턴스에서 Bayes-optimal 예측조차 그것과 약 2/3만 일치한다.

열거 정합성. 여기에 놓치기 쉬운 함정이 있고, 그것을 놓치면 파생된 모든 label이 조용히 오염된다. constraint solver는 "탐색 소진"과 "해를 이미 모은 상태에서 시간제한 도달"을 서로 다른 status로 보고한다. 전자만이 S가 완전함을 보증한다. 후자를 인정하면 잘린 해집합이 대입되어 그로부터 계산된 모든 marginal이 편향된다. 우리 구현은 OPTIMAL과 INFEASIBLE만 인정하고 시간 초과한 instance는 평균에 섞지 않고 폐기한다. 유용한 진단 지표는 평균 열거 시간이 시간제한과 같아지는 것인데, 이는 잘린 instance가 완료로 집계되고 있다는 신호다.

도달 가능한 상태의 표집. depth-d 학습 상태는 d를 균등 범위에서 뽑고, LP 확신도 순서의 앞 d개를 취해, 무작위로 고른 실제 해의 값으로 고정해 만든다. 임의의 값으로 고정하면  $S(F,v) = \emptyset$ 인 상태가 생기는데, 그런 상태는 propagation이 즉시 가지치는 dead node이고 conditional marginal이 정의되지 않으며, branching heuristic이 질문받을 일이 없는 상태다.

### 3.6 Network

instance는 n개 variable node, m개 constraint node, 그리고  $a_{ij}=1$ 인 곳마다 edge를 갖는 bipartite graph로 encoding된다. 현재(축소된) instance의 relaxation 해를  $x^\sim$ , row sum을  $r_i$ 라 하면 node feature는 다음과 같다:

$$\begin{aligned} \text{변수 } j: & (x_j^\sim, \deg(j)/m, |x_j^\sim - \text{round}(x_j^\sim)|) \\ \text{제약 } i: & (b_i/r_i, r_i/n, (b_i - A_i x^\sim)/r_i) \end{aligned}$$

즉 변수에는 relaxation 값 · 열 차수 · fractionality, 제약에는 tightness · scaled row size · relaxation 잔차다. 모두 크기 정규화되어 있으며, 이것이 하나의 파라미터 집합이 임의의 (m,n)에 적용될 수 있게 한다.

두 node 유형 모두 선형 사상으로 h=64차원에 embedding되고, graph attention을 쓰는 양방향 message passing L=4라운드로 정련된다. 각 라운드는 변수→제약, 제약→변수 순이며 각각 LayerNorm과 residual 연결을 갖는다. 이어

2층 head가 변수당 logit 하나를 내어  $p_j^\wedge = \sigma(z_j)$ 를 준다. 모델 파라미터는 21,761개다.

architecture는 의도적으로 작고 단일 head다. 유일한 출력은 목적함수 (4)가 정의하는 양이며, 보조적인 value나 feasibility head가 없다. 탐색은 건전성을 자신의 symbolic 구성요소에서 얻고 network에게는 그 외의 어떤 것도 요구하지 않기 때문이다.

### 3.7 추론: commitment가 아니라 guidance

network는 node당 한 번 호출된다. branching은 다음을 선택한다:

$$j^* = \operatorname{argmax}_j |p_j^\wedge - 1/2|, \text{ 첫 시도 값} = 1[p_{j^*}^\wedge \geq 1/2]$$

즉 network가 가장 확신하는 변수를, 예측한 값으로 먼저 시도한다. 탐색은 complete하게 유지되고 잘못된 예측의 대가는 backtracking뿐이다.

변수 확정은 건전한 deduction만이 수행한다. 두 메커니즘이 있다: constraint propagation, 그리고 반대값 배정이 relaxation조차 infeasible하게 만들 때  $x_j$ 를 고정하는 LP-probing이다. 이 분업은 § 4.5의 측정에서 직접 따라 나온다: 오류의 대가가 시간인 곳에서는 값싼 비건전 predictor가 낮고, 오류의 대가가 정확성인 곳에서는 비싼 건전 절차가 필수이며 대체 불가능하다.

#### Algorithm 1 — Guided backtracking search

```

■■: instance (A,b), guidance model f, ■■ ■■ T
stack ← [(A,b)]
while stack ≠ ∅ and ■■■■ < T:
    (A',b') ← pop(stack)
    if b' < 0 or b' > A'·1: continue // ■■ bound ■■
    A'■ ■■-0 row ■■; ■■ row ■■■■ return ■■ ■■
    propagate; ■■■■ continue
    if propagation■■ ■■ ■■ D■ ■■: push reduce(A',b',D); continue // ■■ 1
    if ■■ ■■ ■■■: ■■■■ return ■■, ■■■■ continue
    p^ ← f(A',b') // ■■ forward pass, ■ ■■
    j* ← argmax |p^_j - 1/2|; v ← 1[p^_j* ≥ 1/2]
    push reduce(A',b',{j*=1-v}), ■■■■ reduce(A',b',{j*=v})
return unresolved

```

### 3.8 비용 모델

설계를 뒷받침하는 비교는 instance 전체 판정 1회당 node 비용이다. node의 자유 변수 수를  $n_c$ 라 하자. network는  $O(n_c+m)$ 개 node를 갖는 graph에 대해 forward pass를 한 번 수행하며, 호출 횟수는  $n_c$ 와 무관하다. LP-probing은 후보 변수마다 하나씩  $n_c$ 회의 relaxation을 푼다.  $n_c \approx 17\sim 20$ 이고 relaxation 한 번이 약 2ms이므로 둘은 한 자릿수 이상 차이 나며, § 4.5에서 19~23배로 측정된다. 학습 컴포넌트가 deduction을 amortize한다는 것은 이런 의미다: probing이 닿지 못하는 결론에 닿는 것이 아니라, 그 결론 대부분에 매 node에서 쓸 수 있는 비용으로 닿는 것이다.

## 4. 실험

### 4.1 인스턴스와 solution multiplicity 통제

인스턴스는 GenHard( $m, n, K$ ) 절차로 생성한다: 밀도 1/2의 무작위 0/1 행렬  $A$ 를 뽑고,  $K=20$ 개의 후보 해를 심어 각각  $b$ 를 계산한 뒤, vertex spread(무작위 objective 방향에서 얻은 relaxation vertex들의 평균 쌍거리)가 최대인 것을 채택한다. vertex spread 최대화는  $S_{LP}$ 를 직접 키우며, market-split 인스턴스를 relaxation 기반 방법에 어렵게 만드는 메커니즘이다 (§ 2.1). 심어진 해의 유일성은 강제하지 않는다.

왜 크기 간  $|S|$ 를 통제해야 하는가. solution multiplicity는 예측 과제의 난이도만이 아니라 성격을 바꾼다.  $|S|=1$ 이면 모든 자유 변수가 DET이고 잘 예측하는 것이 곧 푸는 것이지만,  $|S|$ 가 크면 대부분이 OPEN이고 과제는 퍼진 posterior를 추정하는 것이 된다. 따라서  $|S|$ 를 통제하지 않고 크기를 비교하면 크기와 과제 정체성이 교락된다. 고정된  $n$ 에서  $m$ 이 오르면  $|S|$ 가 줄어들므로, 크기마다  $m$ 을 골라 이를 맞춘다.

| 크기    | $m$ | $m/n$ | $ S $ 중앙값 | root ceiling | conditional ceiling |
|-------|-----|-------|-----------|--------------|---------------------|
| 10×25 | 10  | 0.40  | 23        | 70.4%        | 87.0%               |
| 18×50 | 18  | 0.36  | 19        | 71.3%        | 87.2%               |
| 21×60 | 21  | 0.35  | 10        | —            | —                   |

표 1. 사용한 크기와 맞춰진 solution multiplicity. ceiling (§ 4.3의 Bayes-optimal per-variable 정확도)이 두 학습 크기 간에 1%p 이내로 일치하므로, 변하는 변수는 크기다.

통제의 한계.  $|S|$ 를 맞추면  $m/n$ 이 변한다(0.40→0.35). 고정된  $n$ 에서 둘을 동시에 유지할 수 없고, 과제 정체성을 결정하는  $|S|$ 를 우선했다. 통제는 또한 입자적이다:  $n=60$ 에서  $|S|$  중앙값은  $m=20, 21, 22$ 에 대해 42→12→2로 계단을 이루므로  $m=21$ 이 사용 가능한 최선의 정수이며, 실제 평가셋은 중앙값 10에 27/30이 검증됐다.

열거가 멈추는 지점.  $n=70$ 에서는 목표 multiplicity를 주는  $m$ 에서 120초 내에 열거가 하나도 완료되지 않았다(0/10).  $n=100$ 에서는 장애가 더 근본적이다: 심어진 해가 있음에도  $m \in \{28, 32, 36, 40\}$ 에서 constraint solver가 20초 내에 해를 하나도 찾지 못한다. 그 크기에서 병목은 분기를 잘 고르는 것이 아니라 해를 하나라도 찾는 것이므로 branching 비교가 유익하지 않다. 따라서  $n \leq 60$ 에서 평가하며 이 경계를 한계로 보고한다.

### 4.2 구현과 프로토콜

소프트웨어. relaxation은 PySCIPOpt를 통한 SCIP, 열거와 CP-SAT baseline은 OR-Tools, lattice reduction은 fpylll(LLL, BKZ)로 수행한다. network는 PyTorch와 PyTorch Geometric(GATConv)으로 구현했다.

학습. Adam, learning rate  $10^{-3}$ , 20 epoch, in-tree 상태 10,000개, batch size 1(instance 크기가 달라서), gradient norm clipping 1.0, 정확한 marginal target에 대한 손실 (4). 학습 상태는 depth 0~12에서 균등하게, 원본 instance당 6개씩, 1,760개 instance 풀에서 뽑는다. 평가는 분리된 풀의 440개 instance를 쓴다. 학습은 CPU 코어 1개로 약 20분 걸린다.

스레딩. 타이밍에 민감한 모든 구성요소는 단일 스레드로 실행한다(`torch.set_num_threads(1)`). 이 크기의 graph에서 다중 스레드 BLAS는 역효과다: forward pass가 8스레드에서 11.3ms, 1스레드에서 0.9ms로 측정된다.

탐색 평가. 각 arm은 동일한 instance에 대해 subprocess 종료로 강제되는 인스턴스당 hard wall-clock 예산으로 실행된다. lattice reduction이 in-process 타이머로 중단할 수 없는 blocking native 호출 안에서 실행되기 때문이다. instance는 20코어 머신에서 동시성 6으로 실행한다. 이 낮은 동시성은 의도적이다: wall-clock 예산 비교는 CPU 경합에 민감하며, 더 높은 동시성에서 단독 실행 시 14~15초에 풀리는 instance가 20초 상한을 넘는 것을 관측했다.

통계. 동일 instance에 대한 arm 간 해결율 차이는 McNemar 정확검정으로, 인스턴스 단위 속도 차이는 양측 부호검정으로 검정한다. 검정력이 다를 수 있으므로 단일 총계 대신 둘 다 보고한다.

### 4.3 Bayes ceiling 대비 예측 품질

$S$ 가 정확히 열거되므로 Bayes-optimal per-variable 정확도를 계산할 수 있다:

$$\text{ceil} = (1/|K|) \sum_j \max(p_j, 1-p_j) \quad (5)$$

이는 살아남은 해들의 다수값을 답함으로써 달성되며, (A,b)로부터  $x^*$ 를 예측하는 어떤 결정론적 predictor도 이를 넘을 수 없다. 표 2는 S가 완전히 열거된 held-out  $10 \times 25$  instance 400개에서 여러 predictor를 이에 대조한다.

| Predictor                   | 전체 변수 | top-3 | top-5 | marginal과의 $L_1$ |
|-----------------------------|-------|-------|-------|------------------|
| Bayes ceiling               | 69.9% | 93.1% | 90.5% | 0.0000           |
| 제안 network                  | 68.6% | 87.4% | 85.2% | 0.1261           |
| soft target (4)로 학습         | 67.7% | 87.2% | 85.3% | 0.0977           |
| Multi-head GNN (2.11M 파라미터) | 60.7% | 72.0% | 69.5% | 0.1813           |
| LP relaxation rounding      | 60.6% | 64.5% | 64.1% | 0.3213           |

표 2. 정확한 posterior 대비 root 수준 예측, held-out  $10 \times 25$  instance 400개. top-k는 predictor가 가장 확신하는 k개 변수로 제한한 것이다. multi-head 모델은 selection/assignment/value/feasibility 4개 head를 갖는 2.11M 파라미터 network이고, 제안 network는 21.8K 파라미터다.

두 가지가 관찰된다. 21.8K 파라미터 단일 head network가 ceiling의 1.9%p 이내, top-3 부분집합에서 5.7%p 이내에 도달하는 반면, 97배 큰 multi-head network는 단순 relaxation rounding과 통계적으로 구별되지 않는다(60.7% 대 60.6%). 즉 용량이 구속 조건이 아니며, 출력이 곧 target인 모델이 그것을 여러 head 중 하나로 생산하는 모델을 실질적으로 앞선다. 그리고 심어진 해 대신 정확한 marginal로 학습하면 posterior 추정이 뚜렷이 개선되고( $L_1$  0.1261→0.0977) argmax 정확도는 변하지 않는데, § 3.4의 분석이 소비하는 것이 argmax가 아니라 marginal이므로 이것이 중요하다.

총계가 가리는 것. 표 3은 같은 root 변수들을 § 3.4의 분할로 분해한다.

| root 변수                   | 개수  | 비중    | 부분집합 ceiling |
|---------------------------|-----|-------|--------------|
| DET (거슬러 분기하면 subtree가 빔) | 38  | 6.3%  | 100.0%       |
| OPEN (어느 쪽이든 해에 도달)       | 562 | 93.7% | 67.2%        |
| 전체                        | 600 | 100%  | 69.3%        |

표 3. root 수준 분해, S가 열거된  $10 \times 25$  instance 24개. 총계는 그 94%가 어떤 branching 결정도 backtrack을 유발할 수 없는 변수로 이뤄져 있다.

root에서는 변수의 93.7%가 OPEN이다. 따라서 총계 ceiling 69.9%는 탐색에 영향을 줄 수 없는 예측이 지배하며, 영향을 줄 수 있는 6.3%에서는 정보가 이미 완전하다(100%). 결과적으로 root 수준 정확도는 branching heuristic에게 약한 목적함수이고, 핵심 질문은 정보가 존재하는가가 아니라 무엇이 그것을 탐지하는가이다 — 다음 절의 주제다.

#### 4.4 Depth, 그리고 잔차가 있는 곳

명제 1을 써서, 실제 해에서 뽑은 prefix를 따라 각 depth에서 conditional ceiling과 propagation이 강제하는 남은 변수의 비율을 측정한다(표 4).

| depth | 살아남은  S | conditional ceiling | propagation이 강제 | 제안 network |
|-------|---------|---------------------|-----------------|------------|
| 0     | 25.2    | 70.5%               | 0.0%            | 67.9%      |
| 4     | 4.1     | 82.2%               | 0.4%            | 73.2%      |
| 7     | 1.9     | 92.6%               | 4.1%            | 80.4%      |
| 8     | 1.6     | 93.4%               | 8.2%            | 85.0%      |
| 12    | 1.1     | 98.6%               | 56.7%           | 97.4%      |
| 15    | 1.0     | 99.7%               | 94.6%           | —          |

표 4. depth에 따른 conditional 정보량. n=25, instance 150개, LP 확신도 변수 순서.

중요한 변화는 수준이 아니라 구성에 있다: DET 비중이 root의 6.3%에서 depth 8의 89.0%로 오른다. root에서는 거의 모든 branching 결정이 무해하다 — 양쪽 자식 모두 해를 포함하기 때문이다. depth 8에서는 거의 모든 결정이 틀리면

subtree를 비운다. root 상태 대신 in-tree 상태로 학습하면 depth 6~10 구간에서 정확도가 5.3~11.5%p 개선된다.

표 5는 이 depth들에서 잔차를 분해하고 각 방법이 무엇을 탐지하는지 묻는다.

| depth | %DET  | network (DET) | propagation | LP integrality | LP-probing | network (OPEN) |
|-------|-------|---------------|-------------|----------------|------------|----------------|
| 5     | 61.5% | 90.4%         | 0.9%        | 66.5%          | 62.2%      | 57.9%          |
| 6     | 72.1% | 89.2%         | 1.6%        | 67.3%          | 68.1%      | 52.3%          |
| 7     | 83.0% | 87.9%         | 4.1%        | 71.0%          | 75.0%      | 52.9%          |
| 8     | 89.0% | 89.2%         | 7.5%        | 76.9%          | 79.4%      | 59.7%          |

표 5. 잔차 분해. 4~6열은 ground-truth DET 집합에 대한 recall, 3열과 7열은 각 부분집합에서 network의 argmax 정확도다.

잔차는 전적으로 DET 변수 위에 있다: depth 7에서  $0.83 \times 12.1 \approx 10\%$ 로, conditional ceiling 대비 측정된 12.2%p 미달분과 부합한다. OPEN 변수에서 50% 근처 성능은 결함이 아니라 올바른 거동이다 — 그 변수들은 진정으로 미결정이기 때문이다. 두드러지는 수치는 unit propagation이 논리적으로 결정된 변수의 0.9~7.5%만 탐지한다는 것이다: 정보가 존재하고 연역 가능한데도 가장 값싼 건전 규칙으로는 닿지 않는다. LP-probing은 62~79%를 회수하고, LP integrality와 합치면 82.8~92.0%를 회수한다.

#### 4.5 Node당 비용

DET 변수를 잘 탐지하는 두 방법은 비용에서 크게 갈린다. 표 6이 instance 전체 판정에 대해 단일 스레드로 동일 상태에서 둘을 측정한다.

| depth | network 시간 | DET 정확도 | LP-probing 시간 | 탐지율   | 비율    |
|-------|------------|---------|---------------|-------|-------|
| 5     | 1.59 ms    | 89.9%   | 36.85 ms      | 60.5% | 23.1× |
| 6     | 1.61 ms    | 91.3%   | 35.30 ms      | 68.8% | 22.0× |
| 7     | 1.58 ms    | 88.2%   | 32.74 ms      | 77.9% | 20.8× |
| 8     | 1.59 ms    | 88.5%   | 30.99 ms      | 79.8% | 19.5× |

표 6. instance 전체 판정 1회당 node 비용. probing은 변수당 relaxation을 한 번씩 풀고, network는 단일 forward pass로 전 변수를 답한다.

network는 같거나 더 나은 적중률에서 19~23배 저렴하다. 두 수치는 같은 종류가 아니다: probing의 것은 탐지 recall이며 검사가 발화한 곳에서는 결론이 증명이고 나머지에서는 기권한다. network의 것은 전 변수에 대한 argmax 정확도이며 어디에도 보장이 없다. branching에서는 어떤 변수든 선택해야 하고 기권이 실행 가능한 답이 아니므로, 어디서나 적당한 정확도로 답하는 것이 부분집합을 정확히 증명하는 것보다 우월하다. 확정에서는 순위가 뒤집히고 probing이 대체 불가능하다. 이것이 § 3.7 분업의 실증적 근거다.

#### 4.6 탐색 성능

lattice 단계를 끄고 branching 품질만 격리한 채, wall-clock 예산을 맞춰 네 가지 branching 전략을 비교한다. random은 임의 변수·값을, lp는 가장 정수에 가까운 relaxation 값을 갖는 변수를 그 값으로, lp-probe는 전 자유 변수를 probing해 강제된 것을 건전하게 확정된 뒤 lp처럼 분기하며, model은 제안 network를 쓴다.

| 크기 (예산)      | 전략       | 해결율   | node 중앙값 | 시간 중앙값  | node/s |
|--------------|----------|-------|----------|---------|--------|
| 10×25 (60초)  | random   | 100%  | 1,514    | 0.07초   | —      |
|              | lp       | 100%  | 66       | 0.12초   | —      |
|              | lp-probe | 100%  | 12       | 0.79초   | —      |
|              | model    | 100%  | 22       | 0.08초   | —      |
| 20×50 (300초) | random   | 0%    | —        | —       | —      |
|              | lp       | 100%  | 8,876    | 24.93초  | 376    |
|              | lp-probe | 100%  | 173      | 28.14초  | 6      |
|              | model    | 100%  | 1,676    | 6.37초   | 273    |
| 21×60 (600초) | lp       | 90.0% | 69,015   | 190.76초 | 172    |
|              | model    | 100%  | 12,010   | 47.28초  | 135    |

표 7. branching 전략 비교, lattice 단계 비활성화, 각 칸 30개 instance.

세 가지를 짚는다. 첫째, node 수와 wall-clock이 이 방법들을 정반대 순서로 세운다: lp-probe는 압도적으로 작은 tree를 만들지만(20×50에서 lp 대비 51배 적음) 가장 느리며, 초당 6 node를 처리하는 반면 lp는 376 node를 처리한다. branching heuristic을 tree 크기만으로 평가하는 통상적 관행은 여기서 최악의 방법을 선택하게 된다. 둘째, model이 생산적인 지점에 위치한다: lp 대비 node를 5.3배 줄이면서 node당 비용은 lp보다 낮아 시간 중앙값이 3.9배 낮다. 셋째, 21×60에서 이점이 질적으로 변한다: 양쪽 다 푼 27개 중 20개에서 model이 더 빠르고(중앙값 2.71배, 부호검정  $p=0.019$ ), 총 시간을 6,227초에서 2,421초로 줄인다. 게다가 lp가 예산을 소진한 3개 instance를 각각 41초, 230초, 232초에 추가로 해결한다.

두 효과는 서로 다른 확신도로 보고한다. 속도 이점은 유의하다. 해결율 이점은 방향이 완벽히 일관되되(3-0, lp는 한 개도 이기지 못함) 검정력이 부족하다: 3-0 분할은 McNemar  $p=0.250$ 이 최선이며 유의성에는 5-0이 필요하다.

#### 4.7 크기 transfer

feature가 크기 정규화되어 있고 파라미터가  $(m,n)$ 에 의존하지 않으므로 하나의 network가 임의의 크기에 적용된다. 표 1처럼  $|S|$ 를 맞춘 상태에서 그것이 실제로 전이되는지 검증한다.

| 학습 크기 | 평가 크기 | 해결율  | 시간 중앙값 | node 중앙값 |
|-------|-------|------|--------|----------|
| 10×25 | 10×25 | 100% | 0.08초  | 23       |
| 10×25 | 18×50 | 100% | 2.84초  | 833      |
| 18×50 | 18×50 | 100% | 3.42초  | 1,001    |
| 10×25 | 21×60 | 100% | 47.28초 | 12,010   |
| 18×50 | 21×60 | 100% | 54.75초 | 14,108   |

표 8. Transfer, lattice 단계 비활성화, 행마다 30개 instance. 2~3행과 4~5행은 각각 평가 크기를 고정하고 학습 크기만 바꾼 것이다.

21×60 평가 크기에서, 2.4배 작은 문제인 10×25에서만 학습한 network와 18×50에서 학습한 network는 통계적으로 구별되지 않는다: 전자가 정확히 30개 중 15개에서 더 빠르고, 속도비 중앙값 0.92배, 총 시간이 0.4% 차이(2,924.9초 대 2,914.5초)이며 양쪽 모두 30/30을 푼다. root 수준 예측도 같은 결론을 준다: 10×25 network가 10×25, 20×50, 40×100에서 각각 67.8%, 69.5%, 69.4%를 기록하며 relaxation rounding 대비 우위(+11.0, +7.2, +4.7%p)를 모든 크기에서 유지한다.

이것이 실무적으로 중요한 이유는 label 생성이 구속 조건이기 때문이다. 정확한 marginal은 열거를 요구하는데 이는  $n \approx 60$ 을 넘으면 실패한다. transfer는 label을 얻을 수 있는 곳에서 학습한 network를 그럴 수 없는 곳에 배치할 수 있음을 뜻한다.

#### 4.8 전체 pipeline에서의 배치

표 7은 lattice reduction을 꺼서 branching을 격리한 것이다. 실제 배치에서는 lattice 단계가 먼저 돌고 search는 그 잔여만 본다. 그 단계가 branching heuristic과 무관하므로 모든 arm이 동일한 잔여 집합을 받는다.

| 크기    | 전략         | lattice가 해결 | 잔여를 search가 해결 | end-to-end  |
|-------|------------|-------------|----------------|-------------|
| 10×25 | 양쪽         | 30/30       | —              | 100%        |
| 18×50 | lp / model | 24/30       | 6/6 / 6/6      | 100% / 100% |
| 21×60 | lp         | 16/30       | 12/14          | 93.3%       |
| 21×60 | model      | 16/30       | 14/14          | 100%        |

표 9. Cascade 결과, 21×60에서 600초 예산.

lattice coverage는 크기에 따라 떨어진다 — 30/30, 24/30, 16/30 — 그리고 그 감소가 branching 품질이 중요해질 여지를 만든다. 10×25와 18×50에서는 잔여가 비어 있거나 어느 전략으로도 쉽게 닫힌다. 21×60에서는 14개 instance가 search로 넘어오고, 거기서 학습된 guidance가 14개를 모두 닫는 반면 lp는 12개를 닫아 end-to-end 해결율이 93.3%에서 100%로 오른다.

#### 4.9 한계

Label 생성이 평가만이 아니라 방법 자체를 제한한다. 정확한 marginal은 S의 열거를 요구하는데 이 계열에서는  $n \approx 60$ 을 넘으면 완료되지 않는다. transfer는 더 큰 크기에서의 배치를 허용하지만 거기서의 학습은 허용하지 않는다. solution sampling으로  $p_j$ 를 근사하는 것이 자연스러운 확장이며 미검증이다.

유용한 크기 창이 양쪽에서 막혀 있다. 그 아래에서는 lattice reduction이 전부를 닫아 branching 품질이 무의미하고, 그 위( $n=100$ )에서는 constraint solver가 20초 내에 해를 하나도 찾지 못해 모든 전략에서 search가 실패한다. 우리 시연은  $n=60$ 에 있으며 창이 더 확장된다는 것은 보이지 못했다.

통계적 검정력. 21×60에서의 해결율 이점은 방향상 3-0이지만  $p=0.250$ 이며, 속도 이점( $p=0.019$ )만이 유의하다. 전자를 확정하려면 더 큰 평가셋이 필요하다.

Multiplicity 통제는 근사적이다. 세 크기에서  $|S|$  중앙값이 23, 19, 10이며,  $n=60$ 에서는 연속한 정수  $m$ 에 대해  $42 \rightarrow 12 \rightarrow 2$ 로 계단을 이루므로 더 정밀한 matching이 불가능하다. 평가 instance 30개 중 3개는  $|S|$ 가 미검증이며 그렇게 표시되어 있다.  $|S|$ 를 맞추는 것은  $m/n$ 을 0.40에서 0.35로 이동시키기도 한다.

단일 instance 계열. 모든 결과가 하나의 응용에서 나온 하나의 생성기에 관한 것이다. amortization 논변이 다른 constraint 계열로 전이되는지는 미검증이다.

### 5. 결론

본 논문은 binary linear system을 위한 학습 기반 branching heuristic과, 그것을 지도학습하는 정확한 절차를 제시했다. 두 요소가 방법을 떠받친다. BLS를 partial assignment로 conditioning하는 것은 그것을 축소하는 것과 동일하므로, 평범한 instance로 학습된 하나의 network가 구조 변경 없이 search tree의 모든 node에서 예측한다. 그리고 전수열거가 가능한 크기의 instance에서 정확한 conditional posterior는 학습 target을 공급하는 동시에 어떤 예측이 탐색에 영향을 줄 수 있는지를 식별한다: 살아남은 모든 해가 일치하는 변수만이 backtrack을 유발할 수 있으며, 그 비중은 root의 6.3%에서 depth 8의 89.0%로 오른다.

그 결과 얻어진 heuristic은 solver 안에서 특정한, 그리고 우리가 보기에 올바른 위치를 차지한다. solver가 연역할 수 있는 범위를 넓히지 않는다: network가 잘 예측하는 변수는 정확히 LP-probing이 증명할 수 있는 변수이며, network는 branching에만 쓰이고 확정에는 쓰이지 않는다. 바뀌는 것은 비용이다 — node당 31~37ms 대신 1.6ms, 같거나 더 나은 적응률에서 — 그리고 이것이 선별된 몇 곳이 아니라 매 node에서 guidance를 가능하게 한다. wall-clock 예산을 맞춘 비교에서 21×60 기준 median 2.71배 가속과 node 6.2배 감소를 얻고, 실제 cascade에서는 lattice reduction만으로는 부족해지기 시작하는 크기에서 end-to-end 해결율을 93.3%에서 100%로 올린다. solution multiplicity를 통제하면 2.4배 작은 규모에서 학습한 network가 평가 크기에서 학습한 것과 구별되지 않게 동작하는데, 정확한 지도가 작은 크기에서만 가능하므로 이는 중요한 성질이다.

이것이 시사하는 framing은 학습된 발견이 아니라 amortized deduction이다: 여기서 학습의 기여는 비싼 건전 절차를 어디서나 쓸 수 있을 만큼 값싸게 만드는 것이며, mixed-integer programming에서 branching imitation이 하는 바로 그 역할이다. 가장 분명한 향후 방향은 구속 조건에서 따라 나온다 — 전수열거 없이 conditional marginal을 근사하면 정확한 지도를  $n \approx 60$  너머로 확장할 수 있고, LP-probing 결정을 직접 distillation하는 것이 비용 프로파일이 측정된 지금 두 번째 보완적 target이다.

## 재현성

코드는 proposed\_src/ 아래에 구성요소별로 정리되어 있고, 공용 모듈은 util/에, 각 디렉토리의 ROLES.txt에 모든 파일 설명이 있다. 스크립트는 각자의 디렉토리에서 실행한다.

```
# |S| ■ ■ ■ instance ■ ■ ■ ■ marginal ■ ■ ■
cd proposed_src/v15_bayes
python3 v15_gen_by_m.py --m 10 --n 25 --count 2200 --K 20 \
    --out ../../runs/data/sols_10x25.json

# in-tree ■ ■ ■ ■ (depth 0-12, instance ■ ■ 6)
cd ../v17_conditional
python3 v17_make_conditional_data.py \
    --sols ../../runs/data/full_10x25_train.json \
    --max_depth 12 --per_instance 6 \
    --out ../../runs/data/cond_10x25_train.json

# ■ ■ (CPU 1 ■ ■ ■ ■ 20)
python3 v17_train_conditional.py \
    --train ../../runs/data/cond_10x25_train.json \
    --test ../../runs/data/cond_10x25_test.json \
    --n_train 10000 --n_test 2000 --epochs 20 --target soft \
    --out ../../runs/model_n25

# ■ ■: branching arm x {lattice off, lattice on}
cd ../v22_transfer
python3 v22_cascade_search.py \
    --data_dir ../../instances/v22test_21x60 \
    --tag EVAL --time_limit 600 --block 12 --tries 10 \
    --arms lp model --ckpt ../../runs/model_n25/conditional.pt \
    --ahl off on --jobs 6 --out_root ../../runs/eval
```

모든 생성기는 seed가 고정되어 있다. 공개 checkpoint는 각 96KB다. instance 데이터와 가중치는 배포 대신 위 명령으로 재생성하며, 전체 평가 사이클은 v22\_transfer/run\_v22\_cd.sh에 스크립트화되어 있다.

## 참고문헌

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Amos, B. (2023). Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732.
- Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization*, pages 284–293.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*.
- Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In *Proceedings of the 36th Annual Meeting of the Cognitive Science Society*, pages 517–522.

- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In AAAI Conference on Artificial Intelligence, pages 724–731.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Millidge, B. (2022). Deconfusing direct vs amortized optimization. *비심사 기술 노트*, <https://www.beren.io/2022-09-25-Deconfusing-direct-vs-amortized-optimization/>
- Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S. (2001). Chaff: Engineering an efficient SAT solver. In *Design Automation Conference*, pages 530–535.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1–3):181–199.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations*.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In *International Conference on Learning Representations*.